

CURSO 3: CREACIÓN Y DESARROLLO DE AGENTES DE IA

CLASE 06 • NIVEL 3 - AVANZADO

Clase 06: Arquitecturas RAG (Retrieval-Augmented Generation)

«RAG como Darle al LLM un Libro Abierto con la Información Exacta»

Guía de estudio oficial y manual técnico. Diseñado para dominar la programación en Python mediante modelos mentales rigurosos, diagramas de flujo y código de producción.

Autor & Mentor: William Rodríguez (Wisrovi)

Rol: AI Solutions Architect & Principal Software Engineer

Python: 3.10+ | **Licencia:** MIT | **wisrovi SUITE**

Acerca del Autor y Mentor



William Rodríguez (Wisrovi)

AI Solutions Architect & Principal Software Engineer • Badajoz, España

Ingeniero y arquitecto de software especializado en Inteligencia Artificial Generativa, sistemas multi-agente, Visión por Computador e infraestructuras MLOps de alta disponibilidad. Creador y mantenedor de la suite de software libre wisrovi SUITE en PyPI con más de 26 bibliotecas enfocadas en orquestación de pipelines, caching distribuido y optimización de bases de datos.



GitHub: github.com/wisrovi



LinkedIn: [in/wisrovi-rodriguez](https://in.linkedin.com/in/wisrovi-rodriguez)



DockerHub: hub.docker.com/u/wisrovi



Website: wisrovi.dev

Metodología de Aprendizaje: La Regla de la Bicicleta 🚲

En este programa no creemos en el aprendizaje pasivo. Programar no se aprende memorizando manuales o mirando videos en segundo plano mientras tomas café; se aprende **escribiendo código**, enfrentando errores de sintaxis, depurando variables y viendo cómo responde el intérprete en tiempo real.



El Compromiso Activo del Estudiante

Abre Visual Studio Code en cada sesión. Escribe cada ejemplo con tus propias manos. Cambia los números, rompe el código deliberadamente para ver el mensaje de error de Python, y luego arréglalo. Ese proceso de experimentación es el que construye sinapsis duraderas.

Presentación de la Sesión

Bienvenido/a a la **CLASE 06** del **Curso 3: Creación y Desarrollo de Agentes de IA**. Esta guía técnica está estructurada para proporcionarte las bases conceptuales, arquitectónicas y prácticas indispensables para tu progreso.

Objetivos de la Sesión

Competencia Conceptual: Comprender la división en Chunks, overlap, indexación en bases vectoriales y síntesis con contexto.

Competencia Práctica: Implementar un pipeline RAG completo en memoria para responder preguntas sobre documentos propios.

Estructura del Documento

Sección	Contenido Temático	Objetivo Pedagógico
Pág 4: Fundamento	Teoría, Modelo Mental y Metáfora	Comprensión del concepto
Pág 5: Flujo	Diagrama de Arquitectura de Memoria	Visualización del flujo interno
Pág 6: Código	Implementación en Python 3.10+	Patrones idiomáticos (PEP 8)
Pág 7: Gotchas	Antipatrones y Trampas Comunes	Prevención de bugs en producción
Pág 8: Conclusiones	Cierre de Sesión & Notas de Repaso	Consolidación del aprendizaje
Pág 9: Bibliografía	Fuentes Canónicas y Desafío	Ampliación y autoestudio

Clase 06: Arquitecturas RAG (Retrieval-Augmented Generation)

RAG combina recuperación semántica con generación de lenguaje para que el LLM responda con datos privados sin reentrenamiento.

☀ Metáfora Central: RAG como Darle al LLM un Libro Abierto con la Información Exacta

En lugar de pedirle al alumno que responda de memoria, le permites consultar el capítulo exacto del libro antes de contestar.

Principios Teóricos y Modelo Mental

Ingestión: Carga de documentos -> División en Chunks (ej. 500 caracteres con 50 de solapamiento) -> Embeddings -> Vector Store.

Recuperación y Aumento: Consulta de usuario -> Búsqueda vectorial Top-K -> Inyección en el System Prompt.

⚡ Regla de Oro en Python

Instruye al modelo a responder ÚNICAMENTE basándose en el contexto provisto para eliminar alucinaciones.

Arquitectura y Movimiento de Datos en Memoria

Pipeline RAG de 2 fases: Ingestión Indexada y Consulta en Tiempo Real.



Desglose Paso a Paso del Diagrama

Fase del Flujo	Acción del Intérprete	Estado en Memoria
1. Inicialización	Chunking de documentos y generación de embeddings.	Vectores almacenados en el Vector Store.
2. Evaluación	Recepción de pregunta del usuario y embedding de la query.	Vector de consulta generado.
3. Transformación	Búsqueda de vecinos más cercanos (Top-3 Chunks).	Contexto relevante recuperado.
4. Retorno / Salida	Construcción del prompt aumentado y generación final del LLM.	Respuesta precisa con citas de fuentes.



Visualización Mental

La calidad de un sistema RAG depende en un 80% de la calidad del chunking y la recuperación.

Código de Demostración en Python 3.10+

Pipeline RAG funcional en memoria en Python puro:

main.py (Python 3.10+)

PEP 8 Compliant

```
class MiniRAG:
    def __init__(self):
        self.docs = []

    def indexar(self, texto: str):
        # Simulación de chunking básico
        self.docs.append(texto)

    def recuperar(self, query: str) -> str:
        # Recupera el documento con mayor coincidencia léxica
        palabras = set(query.lower().split())
        mejor_doc = max(self.docs, key=lambda d:
            len(palabras.intersection(set(d.lower().split()))))
        return mejor_doc

    def generar_prompt(self, query: str) -> str:
        ctx = self.recuperar(query)
        return f"Contexto:\n{ctx}\n\nPregunta: {query}\nRespuesta basada estrictamente en el\ncontexto:"

rag = MiniRAG()
rag.indexar("El horario de atención es de Lunes a Viernes de 9:00 a 18:00.")
print(rag.generar_prompt("¿A qué hora abren?"))
```

Análisis de Ingeniería del Código

Indexación modular, recuperación por coincidencia y construcción de prompt enriquecido.

💡 Buena Práctica de Tipado

El uso sistemático de Type Hints (PEP 484) permite a los editores modernos como VS Code activar autocompletado inteligente y detectar errores antes de la ejecución.

Trampas Frecuentes de Depuración (Gotchas)

Peligros del 'Lost in the Middle' en RAG.

⚠ Gotcha Frecuente (Trampa de Principiante)

Inyectar demasiados chunks (ej. 20 chunks) satura el contexto y hace que el LLM ignore la información central.

Comparativa: Antipatrón vs Patrón Recomendado

❌ Antipatrón

```
prompt = f'Contexto:\n{20_chunks_desordenados}'  
# ❌ Degradación de atención
```

✅ Patrón Correcto

```
# Selecciona Top 3 a 5 chunks relevantes y  
reordénalos con un Re-ranker ✅
```

🛡 Consejo de Resiliencia en Producción

Usa técnicas de Hybrid Search (búsqueda léxica BM25 + búsqueda vectorial) para máxima precisión.

Conclusiones Clave de la Sesión

Hemos cubierto los pilares teóricos y prácticos de **Clase 06: Arquitecturas RAG (Retrieval-Augmented Generation)**. El dominio de esta unidad te proporciona una base sólida para afrontar la siguiente semana formativa.

Hitos Alcanzados

Comprensión profunda del modelo mental, capacidad de depuración de gotchas comunes y solidez en la sintaxis idiomática de Python 3.10+.

Notas de Repaso Rápido

Concepto	Punto Clave a Recordar
1. Modelo Mental	RAG como Darle al LLM un Libro Abierto con la Información Exacta
2. Regla de Oro	Instruye al modelo a responder ÚNICAMENTE basándose en el contexto provisto para eliminar alucinaciones.
3. Gotcha Crítico	Injectar demasiados chunks (ej. 20 chunks) satura el contexto y hace que el LLM ignore la información central.
4. Buenas Prácticas	Usa técnicas de Hybrid Search (búsqueda léxica BM25 + búsqueda vectorial) para máxima precisión.

Mensaje del Instructor

¡Felicitaciones por completar esta lección! Recuerda que la constancia y el pedaleo diario en tu editor de código son los que te convertirán en un programador excepcional.

Fuentes Bibliográficas Oficiales

Para profundizar en los estándares formales del lenguaje y sus implementaciones internas, consulta la documentación canónica:

Recurso Oficial	Descripción	Enlace
Documentación Python 3	Especificación canónica y biblioteca estándar	docs.python.org/3/
PEP 8 — Style Guide	Estándar oficial de formateo y estilo	peps.python.org/pep-0008/
Real Python Tutorials	Patrones de desarrollo e ingeniería	realpython.com
Suite wisrovi en GitHub	Paquetes open source de alto rendimiento	github.com/wisrovi

🏆 Desafío Práctico para el Estudiante

🎯 Reto de la Semana

Crea una función de chunking con solapamiento configurable que no corte palabras por la mitad.

🔧 Validación Automatizada

Ejecuta `pytest ejercicios/` en tu terminal de VS Code para verificar automáticamente la validez de tu solución.